

Eloquent Basics

Create, query, update, and delete records with your `Student` model.

Reuse Laravel lesson 4's generated `student-api/`.
No new project is needed.

Learning Goals

- Try Eloquent code interactively with Tinker.
- Create, read, update, and delete records with model methods.
- Build a query, and tell whether a call returns a model, `null`, or a Collection.

Try Code Quickly with Tinker

Tinker is an interactive PHP shell with your Laravel app already loaded.

```
cd student-api          # the folder that contains `artisan`  
php artisan tinker
```

```
use App\Models\Student;  
Student::count();        // number of rows in studentdb.students
```

- Tinker talks to the **real** `studentdb`, the same database as Module 1.
- Work only with the practice record you create below. Type `exit` to leave.

A Record Is an Object

With PDO, a row comes back as an array. With Eloquent, it is a model object.

```
$student = Student::firstOrFail();  
get_class($student);    // App\Models\Student  
$student->name;          // an attribute of this record  
$student->toArray();     // all attributes as an array
```

Instance methods such as `update()` and `delete()` work on this one record.

Class-level calls such as `Student::create()` work on the whole table.

Create One Record

```
$student = Student::create([
    'name' => 'Practice Student',
    'email' => 'orm-' . uniqid() . '@example.com',
    'age' => 20, 'major' => 'CS', 'year' => 1,
]);
$id = $student->id;
```

- The generated email avoids duplicates when you repeat the exercise.
- Every field must be in `$fillable`, or `create()` rejects it.
- `create()` does **not** validate values. Validation belongs in the controller.

Read, Update, and Delete

```
$student = Student::findOrFail($id);  
$student->update(['year' => 2]);  
$student->refresh();  
$student->year;                // 2  
$student->delete();  
Student::find($id);           // null
```

Use the ID returned by `create()` instead of assuming ID 1 exists.

The Module 1 table has no `created_at` or `updated_at`, so the model does not use them.

Build a Query

```
$students = Student::where('year', '>', 1)
    ->where('major', 'Computer Science')
    ->orderBy('name')
    ->get();
```

- `where` and `orderBy` build the query. `get()` runs it.
- `first()` returns one model or `null`.
- `Student::count()` asks the database for the number.

Work with a Collection

```
$students = Student::all();  
$names = $students->pluck('name');  
$count = $students->count();  
$groups = $students->groupBy('major');
```

These operations work on records already loaded into memory.

`Student::count()` instead asks the database, which is faster for large tables.

Model, null, or Collection?

Call	Returns
<code>Student::first()</code>	One model, or <code>null</code>
<code>Student::firstOrFail()</code>	One model, or an exception
<code>Student::find(\$id)</code>	One model, or <code>null</code>
<code>Student::all()</code> , <code>->get()</code>	A Collection (possibly empty)

```
Student::whereRaw('1 = 0')->first();    // null; no rows match
Student::whereRaw('1 = 0')->get();      // empty Collection, not null
```

Check which one you have before you call `->name` on it.

Handle a Missing Record

Inside a Laravel controller that imports `App\Models\Student` :

```
public function show($id)
{
    return response()->json(Student::findOrFail($id));
}
```

Laravel converts an uncaught `ModelNotFoundException` to HTTP 404.

A custom `catch` is only needed for a custom response.

Send `Accept: application/json` when testing API errors.

Practice

1. Create a record and keep its generated ID.
2. Query it, update it, and verify the stored value.
3. Delete it and confirm `Student::find($id)` returns `null`.
4. Compare `Student::all()->count()` with `Student::count()`.
5. Explain when you get a model, `null`, and a Collection.